

Testing AI Systems (AI Agents, Chatbots & RAG)

Interview Questions

Testing AI Systems (AI Agents, Chatbots & RAG) Interview Questions

Rahul Shetty Academy - Your one stop shop to master Test Automation

Section 1 — Foundations & Why AI Testing Is Different

Q1. Why can't you use traditional assert statements to test an AI agent's output?

LLMs are non-deterministic — the same input produces differently worded but equally valid outputs on every run. A hardcoded string assertion will fail even when the agent is working correctly. You need semantic evaluation, not string matching.

Q2. What is an "Eval" and how is it different from a traditional test?

An eval measures output quality on a scale of 0.0 to 1.0 against a defined criterion. Traditional tests are binary pass/fail based on exact output. Evals use a judge LLM to score semantic quality and you define a threshold that separates acceptable from unacceptable.

Q3. What is a Judge LLM and why is it needed?

A Judge LLM is a separate model whose job is to read the agent's output and score it. It is needed because evaluating semantic quality — did the agent complete the task? is the answer relevant? does it hallucinate? — requires the same language understanding that produced the output. Rule-based code cannot do this.

Q4. Why should the Judge LLM be from a different provider than the agent being tested?

To avoid self-evaluation bias. An LLM tends to rate its own outputs more favourably than outputs from other models. Using a different provider (e.g., agent on Claude, judge on GPT-4o) makes the scoring more objective and trustworthy.

Q5. What is a "Golden" in DeepEval?

A Golden is one test scenario. It contains the input to send to the agent and optionally expected values such as expected output, expected tools, or

retrieval context. A collection of Golden forms an EvaluationDataset.

Q6. What is a "threshold" in a DeepEval metric and how do you decide what to set it to?

The threshold is the minimum score (0–1) for the metric to pass. Setting it is a design decision. Quality metrics typically use 0.7, efficiency metrics 0.5, and safety metrics should be higher in production. You should also build in margin to absorb the judge LLM's own scoring variance across runs.

Q7. What is the difference between a deterministic metric and an LLM-judged metric? Give one example of each.

A deterministic metric calculates the score using exact logic with no LLM involved. Example: ToolCorrectnessMetric — it compares actual tool names against expected names, a simple set check. An LLM-judged metric sends the output to a judge model for semantic scoring. Example: TaskCompletionMetric — there is no formula for "did the agent complete the job," only a judge can decide.

Section 2 — Agent Evaluation

Q8. What does TaskCompletionMetric measure and what inputs does it need?

It measures whether the agent successfully completed the user's stated goal. It needs the input and actual output (and optionally the full trace). It does NOT require an expected output — the judge infers what "success" means from the input itself.

Q9. What does ToolCorrectnessMetric measure and how does it work internally?

It checks whether the agent called the correct tools during its run. Internally it compares the set of actual tool names called against the set of expected tool names from the Golden. It is deterministic — no LLM judge is involved.

Q10. Can a golden PASS TaskCompletionMetric but FAIL ToolCorrectnessMetric? Give a scenario.

Yes. The agent might reach the right answer using an unexpected combination of tools. For example, if the expected tool is `get_refund_policy` but the agent called `get_order_status` and still gave a correct refund answer from cached context — task is complete, tool name doesn't match.

Q11. What is AnswerRelevancyMetric and when would you use it over TaskCompletionMetric?

AnswerRelevancyMetric checks whether the response directly addresses the user's question. TaskCompletionMetric checks whether the user's broader goal was accomplished. Use AnswerRelevancy when you want to catch off-topic or padded responses. Use TaskCompletion when you want end-to-end success of the full agent workflow.

Q12. What does PromptAlignmentMetric test and what do you pass to it?

It checks whether the agent's response follows the instructions in its system prompt. You pass the system prompt instructions as a list of strings. The judge then checks whether the response respects those instructions — for example, keeping replies short when instructed to.

Q13. What does StepEfficiencyMetric measure and why is its threshold usually lower than other quality metrics?

It measures whether the agent completed its task using the minimum number of steps. The threshold is typically 0.5 rather than 0.7 because agents rarely take the theoretically optimal path — extra reasoning or redundant tool calls are common, so the bar is relaxed to avoid false failures.

Section 3 — Tracing and Instrumentation

Q14. What is "tracing" in the context of DeepEval and why is it important?

Tracing is capturing every internal step of the agent's execution — LLM calls, tool calls, retrieved chunks — as structured spans. It is important because many metrics (ToolCorrectness, StepEfficiency, ContextualPrecision) need to look inside the agent's execution, not just at the final output.

Q15. What does @observe do and where do you place it?

@observe creates the outer trace span that DeepEval uses to record the agent's execution. You place it on the wrapper function that calls your agent — not inside the agent itself. Everything the agent does while inside that function is captured as child spans.

Q16. What does CallbackHandler do in the LangChain integration?

CallbackHandler hooks into LangChain's callback system and automatically captures every LLM call, tool call, and chain step as a DeepEval span. You

create one instance and pass it via `config={"callbacks": [deepeval_callback]}` on `.invoke()`.

Q17. Why do you need to call `update_current_trace` inside the `@observe-wrapped` function? What would happen if you didn't?

DeepEval reads `expected_tools`, `expected_output`, and `retrieval_context` from the trace object — not directly from the Golden. If you don't copy them in via `update_current_trace`, metrics like `ToolCorrectnessMetric` and `ContextualPrecisionMetric` won't find the expected values and will fail or produce incorrect scores.

Q18. Why do safety metrics (Bias, Toxicity, PII) not require tracing but `ToolCorrectnessMetric` does?

Safety metrics only read the final text response — they ask "is this output biased/toxic/PII-leaking?" and need nothing else. `ToolCorrectnessMetric` needs to know which tools were called during execution, which is information that only lives inside the agent's trace, not in the final text.

Q19. What is the difference between `evaluate()` and `evals_iterator()`?

`evaluate()` receives pre-built test case objects (like `LLMTestCase`) and runs scoring on them. `evals_iterator()` iterates over Goldenes one at a time and is designed to be used in a loop where you run the agent live inside each iteration — it feeds each golden into the active trace context so `update_current_trace` and `get_current_golden` work correctly.

Section 4 — GEval and Custom Metrics

Q20. What is GEval and what problem does it solve that pre-built metrics can't?

GEval is a general-purpose metric where you write your own evaluation criteria in plain English. It solves the problem of domain-specific quality requirements that no pre-built metric covers — for example, checking legal compliance language, brand tone adherence, or specific factual accuracy rules.

Q21. What are `SingleTurnParams` and why do you need to specify them in GEval?

`SingleTurnParams` tell the judge LLM which parts of the test case to read when scoring — for example `INPUT`, `ACTUAL_OUTPUT`, `EXPECTED_OUTPUT`. Without specifying them, the judge wouldn't know what context to consider, and

evaluation would be unreliable.

Q22. A team wants to evaluate whether their agent correctly summarises a policy document against a known expected output. Which metric would you recommend and why?

GEval with a Correctness criteria specifying that "the actual output must convey the same factual information as the expected output, with minor wording differences acceptable." This is better than a pre-built metric because it lets you precisely define what "correct summary" means for this domain.

Section 5 — RAG Evaluation

Q23. What are the four main failure modes in a RAG system and which DeepEval metric addresses each?

1. Hallucination — response contradicts retrieved docs → FaithfulnessMetric
2. Poor retrieval ranking — relevant doc ranked last → ContextualPrecisionMetric
3. Incomplete retrieval — relevant doc not retrieved at all → ContextualRecallMetric
4. Off-topic response — right docs retrieved but answer wanders → AnswerRelevancyMetric

Q24. What is the difference between ContextualPrecisionMetric and ContextualRecallMetric?

Precision measures whether the retrieved documents that matter are ranked at the top (ranking quality). Recall measures whether all the information needed to answer the question was actually retrieved (coverage). Both require expected_output. You can have high precision but low recall (right ranking, missing docs) or vice versa.

Q25. FaithfulnessMetric and AnswerRelevancyMetric are often used together in RAG. What does each check?

Faithfulness checks if the response is grounded in the retrieved documents — catching hallucination. AnswerRelevancy checks if the response actually addresses what the user asked. A response can be faithful (only says things in the docs) but still irrelevant (answers a different question). Both are needed for complete RAG quality coverage.

Q26. Which RAG metrics require `expected_output` and which don't? Why the difference?

`ContextualPrecisionMetric` and `ContextualRecallMetric` require `expected_output` because they need a ground-truth answer to evaluate ranking and retrieval completeness against.

`FaithfulnessMetric` and `AnswerRelevancyMetric` do NOT require `expected_output` — `Faithfulness` compares the response against the retrieved context, and `AnswerRelevancy` compares the response against the input question.

Section 6 — Multi-Turn Chatbot Evaluation

Q27. What is a `ConversationalTestCase` and how is it different from `LLMTestCase`?

`ConversationalTestCase` holds a sequence of `Turn` objects representing a multi-turn conversation. `LLMTestCase` holds a single input/output pair.

Chatbots need `ConversationalTestCase` because their correctness depends on what happened across multiple turns, not just one exchange.

Q28. What does `KnowledgeRetentionMetric` test in a chatbot?

It tests whether the chatbot correctly remembers and uses information shared by the user in earlier turns. For example, if the user shared their order ID in Turn 1, the chatbot should correctly reference it when answering a status question in Turn 3 without asking again.

Q29. What is the difference between `TurnRelevancyMetric` and `ConversationCompletenessMetric`?

`TurnRelevancyMetric` evaluates each individual assistant response — is this specific reply relevant to the message that prompted it?

`ConversationCompletenessMetric` evaluates the conversation as a whole — did the entire interaction successfully resolve the customer's goals? A chatbot can pass turn-level relevancy but still fail completeness if the overall problem is never solved.

Q30. Why is `ToolCorrectnessMetric` not usable with `ConversationalTestCase`?

`ToolCorrectnessMetric` is designed to work with `LLMTestCase` and checks tool names against a single expected list. `ConversationalTestCase` operates on a sequence of `Turn` objects across multiple exchanges. The type mismatch makes

them incompatible. Instead, tool quality in multi-turn conversations is assessed via ConversationalGEval with MultiTurnParams.TOOLS_CALLED.

Q31. What is ConversationalGEval and how is it different from regular GEval?

ConversationalGEval is designed for multi-turn conversations. It uses MultiTurnParams to specify which aspects of each turn (ROLE, CONTENT, TOOLS_CALLED) the judge should consider. Regular GEval uses SingleTurnParams and works on a single input/output pair. ConversationalGEval evaluates the quality of the full conversation flow, not individual responses.

Section 7 — Safety Metrics and Synthesizer

Q32. What do BiasMetric, ToxicityMetric, and PIILeakageMetric each check?

- BiasMetric — whether the response contains unfair stereotypes or prejudiced language about demographic groups.
- ToxicityMetric — whether the response contains harmful, offensive, or abusive language.
- PIILeakageMetric — whether the response exposes personally identifiable information like SSNs, emails, or phone numbers.

Q33. Why are safety metric thresholds set lower (0.5) in the course files compared to quality metrics (0.7)?

Safety metric scores can be noisier — the judge LLM may vary more in how it scores subtle bias or mild toxicity. The 0.5 threshold is used as a course default for demonstration purposes. In production, safety thresholds should be much higher, as even a small percentage of biased or toxic responses from a deployed system is unacceptable.

Q34. What is the Synthesizer in DeepEval and why would you use it instead of writing Goldens manually?

The Synthesizer auto-generates Goldens from source documents like policy files. It reads the documents and creates realistic, diverse inputs with expected outputs. It saves time, scales easily when documents change, and surfaces test scenarios a human author might not think of — improving coverage without manual effort.

Section 8 — Architecture and Design Questions

Q35. Your team is building a tool-calling agent. Which metrics would you

include in your baseline evaluation suite and why?

A strong baseline would include:

- TaskCompletionMetric — end-to-end goal completion
- ToolCorrectnessMetric — correct tool selection
- AnswerRelevancyMetric — response stays on topic
- PromptAlignmentMetric — agent follows system prompt instructions

This covers the complete agent loop: intent → tool selection → response quality → instruction compliance.

Q36. You are asked to evaluate a RAG agent for a medical information platform.

What metrics would you choose and what thresholds would you set?

- FaithfulnessMetric at 0.9+ — hallucination in medical context is dangerous
- ContextualPrecisionMetric at 0.8 — wrong ranking can surface wrong medical info first
- ContextualRecallMetric at 0.8 — missing a relevant document could mean missing critical facts
- AnswerRelevancyMetric at 0.8 — off-topic medical responses can mislead patients
- PIILeakageMetric at 0.9+ — patient data must not leak

Higher thresholds across the board — the cost of failure is higher than in a retail support context.

Q37. A colleague says "our agent passes all evals so it must be production-ready." What would you push back on?

Several things: evals are only as good as your Golden dataset — if the goldens don't cover edge cases, the agent can still fail on them in production. The judge LLM is itself non-deterministic and imperfect. Passing at threshold 0.7 means up to 30% of quality is still below perfect. Safety evals at threshold 0.5 are particularly loose. Evals measure specific quality dimensions — you still need load testing, latency testing, and human review of borderline cases.

Q38. When would you choose GEval over a pre-built metric like AnswerRelevancyMetric?

When the quality dimension you care about is domain-specific and not covered by any pre-built metric. For example, checking legal compliance, brand voice,

or specific factual accuracy criteria. AnswerRelevancyMetric has a fixed definition of "relevant" baked in by DeepEval. GEval lets you write exactly what "good" means for your use case in plain English.

Q39. Explain the full flow of what happens when `evals_iterator()` runs a single Golden through the evaluation pipeline.

1. `evals_iterator` yields the Golden and makes it available via `get_current_golden()`
2. Your code calls the agent with `golden.input`
3. The `@observe` decorator creates an outer trace span
4. `CallbackHandler` captures every LangChain LLM/tool step as child spans
5. `update_current_trace` copies `expected_tools` / `expected_output` from the Golden into the trace
6. The agent returns its response
7. DeepEval reads the completed trace and builds an `LLMTestCase` internally
8. Each metric scores the test case — deterministic ones by set comparison, LLM-judged ones by calling the judge model
9. Scores are compared to thresholds and pass/fail is recorded

Q40. What is the difference between how tracing is set up for the LangChain-based agent (`agent_instrumented.py`) versus the direct OpenAI chatbot (`chatbot.py`)?

The LangChain agent uses `CallbackHandler` which hooks into LangChain's callback system to auto-capture all spans, plus `@observe` for the outer trace. The OpenAI chatbot manages its own tool-call loop manually with direct API calls — there is no LangChain to hook into, so tracing is either not needed (for conversational metrics that only need Turn objects) or would need to be added manually per step. Conversational metrics work with the Turn objects collected in the test file, bypassing the need for deep execution tracing.

Total: 40 interview questions across all evaluation layers — concepts, agent, tracing, GEval, RAG, chatbot, safety, and architecture.